From a Phone Scan to a Walkable Web Digital Twin: an End-to-End 3D Gaussian Splatting Pipeline for Indoor Spaces

Team ID: [ENTER TEAM ID] · **Team member:** Palak Sharma · **Round:** 1 — 3D Reconstruction & Roaming
Live web viewer: https://superspl.at/scene/0681979d · **Code:** [GitHub repository URL] · **Video:** [Google Drive video URL]

1. Overview

Goal. Turn an ordinary smartphone capture of a real indoor space into a photorealistic 3D digital twin that anyone can open from a link and roam freely like a video game — with no installation, no manual 3D modelling and no GPU workstation (the team's laptop has only integrated graphics).

Approach. The space is represented with 3D Gaussian Splatting (3DGS). An iPhone running Scaniverse captures and reconstructs the scene on-device following our capture SOP (Section 4); the open-source SuperSplat Editor aligns and prepares the model in the browser; the published scene is compressed to the SOG format, served from a CDN and rendered by the open-source SuperSplat/PlayCanvas web viewer with orbit, free-fly (WASD + mouse) and collision-aware walk modes. A 5 cm collision voxel octree is generated from the same Gaussians — the geometric foundation for obstacle-aware path planning in Round 2.

Result. A 463,573-Gaussian twin of a home (entrance corridor + living room) that loads as a 7.85 MB file — 14.6× smaller than the raw 115 MB reconstruction — and runs interactively on an integrated-GPU laptop: https://superspl.at/scene/0681979d

2. System Architecture

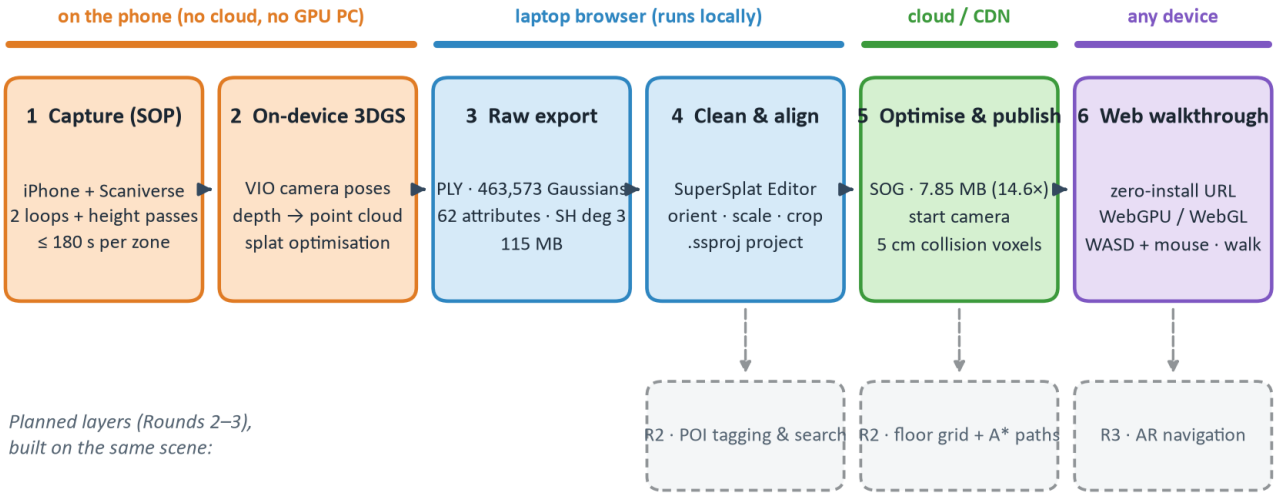


Figure 1. End-to-end pipeline. Each stage hands over an open, inspectable file (PLY → .ssproj → SOG + voxel octree), so any stage can be replaced independently (e.g., on-device training by cloud training for very large venues).

- **Compute where it is cheapest.** Reconstruction runs on the phone's GPU/Neural Engine; editing runs locally in the browser (WebGPU on Intel integrated graphics); heavy compression and voxelisation run server-side; viewing runs on any client.
- **Open formats at every boundary.** PLY for interchange, .ssproj for a reproducible editing project, SOG for web delivery and a sparse voxel octree for collision — all documented and readable by our own scripts (repository /scripts).
- **Operable by non-experts.** Every stage is a GUI action; the only skill an operator needs is the capture SOP.
- **Built for the next rounds.** The same scene, coordinate frame and voxels are reused for POIs and A* navigation (Round 2) and AR relocalisation (Round 3).

3. Why 3D Gaussian Splatting? (first principles)

A remote visitor needs two things at once: the twin must look like the real place — glossy tiles, fabric, lamps, view-dependent highlights — and it must render at interactive frame rates inside a browser. Classical photogrammetry produces a textured mesh: fast to draw, but lighting is baked into textures, and low-texture walls or glossy floors break feature matching. Neural Radiance Fields (NeRF) are photoreal but query a neural network for every ray sample, which is too slow for a browser. 3DGS keeps NeRF's key idea — optimise a scene so that its renders reproduce the input photos — but replaces the network with millions of explicit, differentiable 3D Gaussians that can be rasterised.

Each Gaussian i has a centre μ_i, a covariance $\Sigma_i = R_i S_i S_i^T R_i^T$ (unit quaternion → rotation R_i; per-axis scales S_i), an opacity α_i and spherical-harmonic (SH) colour coefficients that make colour depend on viewing direction. Gaussians are projected to 2-D, depth-sorted and alpha-composited front to back: $C = \Sigma_i\, c_i\, \alpha'_i \prod_{j<i} (1 - \alpha'_j)$, where α'_i is α_i weighted by the projected 2-D Gaussian at the pixel. Parameters are optimised by gradient descent on the photometric loss $L = (1 - \lambda)\cdot L_1 + \lambda \cdot L_D\text{-}SSIM$ ($\lambda = 0.2$ in Kerbl et al., 2023), while adaptive density control clones/splits Gaussians in under-reconstructed regions and prunes transparent ones.

Property	Textured mesh (photogrammetry)	NeRF	3D Gaussian Splatting (ours)
Photorealism, view-dependent effects	Lighting baked; highlights lost	High	High (SH degree 3)
Low-texture walls, glossy floors	Holes / noisy geometry	Better	Better (mirrors still create 'virtual rooms')
Real-time in a web browser	Yes	Difficult	Yes (rasterised, GPU-sorted)
Geometry for collision / navigation	Native	Must be extracted	Derived by voxelising Gaussians (Sec. 6)
Capture → model on a phone	Usually cloud / PC	Needs a GPU server	Minutes, fully on-device

4. Capture SOP (Standard Operating Procedure) v1.0

3DGS can only reconstruct what the camera saw from enough different viewpoints, with sharp images and accurate camera poses. Every rule below protects one of these three inputs — coverage, sharpness, pose accuracy; the reason is given in italics. **Equipment:** any iPhone (A12 or newer) or Android phone with the free Scaniverse app (LiDAR optional); no tripod, PC or GPU. One operator, ≈ 5 minutes per room.

1. **Prepare the space (≈ 2 min).** Switch on all lights and keep them fixed; set curtains/doors to their final state; switch off ceiling fans; keep people and pets out of view; clear loose clutter from walkways. *Why: 3DGS assumes a static scene — anything that moves or flickers becomes blur or 'floaters'; walkway clutter would also become an obstacle for navigation.*
2. **Plan zones.** Split the venue into zones that can each be scanned in ≤ 180 s (one room or a ~5 m corridor segment); adjacent zones overlap by ≥ 1 m at doorways. *Why: Scaniverse warns that splat scans longer than ~180 s risk processing failure; overlap lets zones be aligned and merged later.*
3. **Loop 1 — perimeter, looking inward.** Start in a corner with the phone at chest height (~1.3 m, the eye height of the viewer's walk mode) and walk slowly along the walls (≈ half normal walking speed), keeping the room centre in frame. *Why: parallax from a moving camera is what lets the tracker triangulate depth and poses.*
4. **Loop 2 — centre, looking outward.** From the middle of the room turn slowly through 360°, then sweep each wall, corner and doorway. *Why: walls seen only at grazing angles reconstruct poorly.*
5. **Height passes.** One low pass (~0.5 m, tilted up) along furniture and floor edges, and one high pass (above head, tilted down) for table tops and the ceiling. *Why: furniture legs and floor boundaries define the walkable area used for Round 2 path planning; a single height leaves holes.*
6. **Movement rules.** Move slowly and smoothly; never spin quickly; never fill the frame with a blank wall or a bright lamp from close range (< 0.5 m); pass through doorways slowly and look back once. *Why: motion blur is the most common failure; textureless or saturated frames make pose tracking drift.*
7. **Process and check on site.** Run Scaniverse's on-device splat processing (a few minutes, no upload), then orbit the preview; if any region is blurred, has holes or floating blobs, rescan that zone immediately. *Why: revisiting a venue costs far more than a 3-minute rescan.*
8. **Export and record scale.** Export the splat as a 3DGS .PLY (master copy) and transfer it to the laptop; tape-measure one real distance on site (e.g., a door width) and note it. *Why: the reference length fixes exact metric scale, which walk-mode collision and path lengths depend on.*

Scaling to large venues (e.g., an LHC floor in Round 3). Two operators scan zones in parallel; zones are aligned and merged in SuperSplat (multi-splat alignment) or, for the highest quality, a continuous walkthrough video is posed with COLMAP/GLOMAP and trained with an open-source trainer (Brush/gsplat) on a rented cloud GPU — the downstream pipeline stays unchanged.

5. Reconstruction (this submission)

Dataset. One continuous live scan with Scaniverse on an iPhone, covering the entrance corridor and living room of the participant's home (file Indirapuram.ply). Scaniverse's on-device pipeline uses visual-inertial tracking (ARKit) for camera poses with real-world scale, then aligns frames, estimates depth, initialises a point cloud and optimises the Gaussians — entirely on the phone, so no GPU workstation or cloud credits were needed. The exported PLY stores, per Gaussian:

Attribute block	PLY properties	Count	Meaning
Position	x, y, z	3	Gaussian centre μ
Normal	nx, ny, nz	3	Unused (all zero) — dropped during cleaning
Base colour	f_dc_0..2	3	SH degree-0 (DC) RGB coefficient

View-dependent colour	f_rest_0..44	45	SH degrees 1–3 (15 coefficients × RGB)
Opacity	opacity	1	logit(α)
Scale	scale_0..2	3	log of per-axis standard deviation
Rotation	rot_0..3	4	unit quaternion (w, x, y, z)
Total		62	62 float32 = 248 bytes × 463,573 Gaussians ≈ 115 MB

6. Post-processing, Optimisation and Publishing

- **Alignment (SuperSplat Editor, local browser).** The PLY is imported (File › Import), the floor is levelled to the grid and the model is positioned and scaled; the editable state is saved as a project (scene.ssproj, 113.6 MB) and a cleaned PLY is exported (59 attributes; unused normals removed). Everything runs client-side with WebGPU on integrated graphics.
- **Web compression (SOG).** Publishing converts the splat to SOG: positions are log-transformed and quantised to 16 bits per axis, rotations to 26-bit 'smallest-three' quaternions, scales and colours to 256-entry codebooks, and higher-order SH coefficients are palette-clustered; all data are stored as lossless WebP textures. Result: 7.85 MB (14.6× smaller than the raw PLY) while keeping all 463,573 Gaussians and SH degree 3.
- **Viewing experience (SuperSplat Studio).** A start camera is placed inside the corridor (85° field of view), so visitors land inside the home instead of outside the model.
- **Collision for walk mode.** splat-transform v3.10.3 voxelises sufficiently opaque Gaussians into a 5 cm grid, flood-fills the navigable interior from a seed point and stores it as a sparse voxel octree (500 nodes, 2.9 kB). The viewer moves a capsule (1.5 m tall, 0.2 m radius, eye height 1.3 m, gravity 9.8 m/s²) against these voxels, so the visitor walks on the floor and is stopped by walls and furniture instead of flying through them.

7. Web Viewer — Zero-install Free Roaming

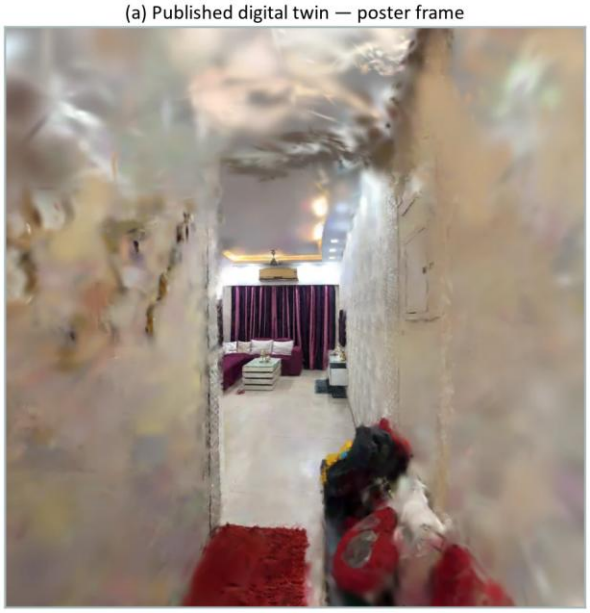
The twin opens from a plain URL on desktop or mobile, with no plug-in or app: https://superspl.at/scene/0681979d (full-screen: https://superspl.at/s?id=0681979d). Renderer: PlayCanvas engine (WebGPU with automatic WebGL 2 fallback); Gaussians are depth-sorted every frame and drawn as screen-space ellipses, and the 7.85 MB SOG payload is decoded on the GPU. Verified on the team's Lenovo IdeaPad Slim 3 (Intel Core Ultra 5, Intel integrated graphics, no discrete GPU).

Input	Action
1 / 2 / 3	Orbit / Fly / Walk mode (walk = gravity + collision with walls and furniture)
W A S D + mouse drag	Move and look around in first person (pressing WASD enters first-person automatically)
Q / E	Move down / up (fly mode)
R · F · H	Reset view · Frame the scene · Show controls help
Touch (phones)	Drag to look around, pinch to move

8. Results

Metric	Value
Capture → reconstruction	iPhone + Scaniverse, single on-device live scan (no cloud processing, no PC GPU)
Gaussians	463,573 (SH degree 3: 48 colour coefficients each)
Raw reconstruction (PLY)	115.0 MB · 62 attributes per Gaussian
Published web asset (SOG)	7.85 MB · 14.6× smaller · all Gaussians kept
Collision	5 cm sparse voxel octree · 2.9 kB
Gaussians inside the home volume	96.1 % (remaining 3.9 % = distant floaters / background)
Viewer requirements	Any modern browser; integrated GPU sufficient; zero install

(a) Published digital twin — poster frame



(b) Top-down projection: floor colour + walls/furniture density

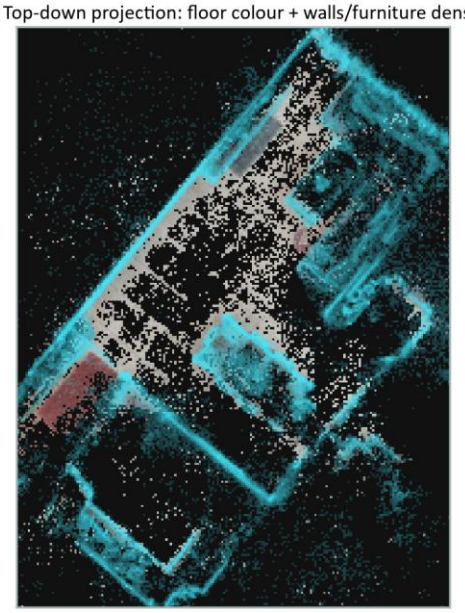


Figure 2. (a) Poster frame of the published twin. (b) Top-down projection of the Gaussians (method in repository scripts/topdown_map.py): floor colour and wall/furniture density (cyan). This 2-D map is the prototype of the occupancy grid for Round 2 path planning.

9. Limitations and Next Steps

- **Metric scale.** The editor applied a uniform scale of 0.326, so one scene unit is not yet exactly one metre. Before Round 2 we calibrate with SuperSplat's Measure tool against a taped door width and regenerate collision — path lengths and the walk-mode capsule are in metres.
- **Floaters.** ≈ 3.9 % of Gaussians lie far outside the home (background through windows, noise). They will be removed with box/sphere selection or splat-transform's cluster filter, shrinking the file further.
- **Coverage.** Single-zone scan: surfaces seen only at grazing angles (behind the sofa, under tables) blur when viewed closely — SOP steps 4–5 address this. Mirrors create virtual geometry and will be masked or deleted.
- **Round 2 — POIs and navigation.** POIs stored as JSON {name, tags, 3-D position, camera pose}; selected by ray-casting against the voxel octree; searched by fuzzy text match with a camera fly-to. Walkable floor cells (free voxels with solid floor below and ≥ 1.8 m head-room) form a 2-D grid inflated by the body radius (0.25 m); A* with an octile heuristic plus path smoothing gives the route, drawn as arrows on the floor in our own web app embedding the open-source @playcanvas/supersplat-viewer.
- **Round 3 — AR (bonus).** A WebXR/ARKit session on the phone is relocalised into the twin (visual positioning or printed QR anchors at known POIs) and arrows are drawn along the same A* path.

10. Tools, Licences and References

- **Scaniverse** (Niantic Spatial) — free mobile app used for capture and on-device 3DGS reconstruction.
- **SuperSplat Editor, SuperSplat Studio/Viewer, PlayCanvas Engine, splat-transform** — PlayCanvas Ltd., open source under the MIT licence; hosting on superspl.at.
- **Python 3, NumPy, Matplotlib, Pillow** — analysis scripts and figures (open-source licences).
- B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," ACM Transactions on Graphics 42(4), SIGGRAPH 2023.
- PlayCanvas developer documentation (developer.playcanvas.com): SOG format specification, Voxel format specification, Collision mesh generation guide, SuperSplat user manual.
- **AI assistance:** GitHub Copilot helped draft this document and the analysis scripts; capture, reconstruction, editing and publishing were performed by the team.